

Take-Home Task — "Shelfie": Bookshelf → Library Inventory

- **Role:** Full Stack Developer (AI & Computer Vision)
- **Effort:** Designed to take about 8 hours
- **Deadline:** 48 hours from the time this was sent to you
- **Presentation:** 30 minutes, scheduled separately

About this exercise

This is a made-up assignment, chosen deliberately at random. It is not part of our product, not on our roadmap, and not something anyone here is building. We picked a bookshelf because it's a domain nobody has a head start in and nobody has strong opinions about, while still exercising the same skills the role needs.

Nothing you write here will be used by us. The repository is yours to keep, reuse, or put on your portfolio.

Ground rules

The task is scoped to roughly 8 hours of work, and you have 48 hours to deliver it. The extra time is there so you can fit the work around the rest of your life — sleep on a problem, start in the evening, take a day off in the middle. It is not there to turn 8 hours of work into 48.

Once you send us the repository link, stop committing. We compare the repository at the presentation against what you submitted, and any commits made after the deadline count against you. If something is broken at the deadline, leave it broken and tell us about it in the README. That is a much better outcome than a repository we can't trust.

We would rather see a small thing that works and that you can defend than a large thing that half-works.

Cutting scope well is part of what we're grading. Tell us in the README what you cut and why.

You may use any AI coding tools you like. We use them too. Assume you will be asked to justify any line in the repository.

What to build

A mobile app that turns a photo of a bookshelf into a structured personal library.

The flow:

1. User takes or picks a photo of a bookshelf in an Expo app.
2. The photo goes to a Django REST API.

3. The backend uses a **pretrained local vision model** to find the individual book spines in the image.
4. The backend uses a **hosted vision-language model** to read title and author off the spines.
5. Each read is matched against **your catalog** (see below) to a canonical catalog entry, with a confidence score.
6. The app shows the result. High-confidence matches can be added directly. Low-confidence and unmatched books go to a review step where the user confirms, corrects, or discards them.
7. Confirmed books persist to the user's library, viewable as a list.

Stack — not negotiable

Layer	Requirement
Frontend	React Native + Expo
Backend	Django + Django REST Framework
Local model	Any pretrained , off-the-shelf model. CPU inference.
Vision-language model	Your choice of hosted provider
Database	SQLite is fine
Deployment	Not required. We must be able to run it on our own machine by following your README.

Do not train or fine-tune anything. You don't have the time, and it isn't what this task is testing. Off-the-shelf weights only.

API keys: ask us and we will issue you a spend-capped key. Or use your own. Either is fine, and neither is scored.

The catalog — you build it

Your app matches against a catalog of canonical books. **Building that catalog is part of the task.** Ship it as `catalog.csv` in the repository.

Requirements:

- **At least 100 entries**, with at minimum a title, an author, and a column for alternate titles or synonyms. Add any other fields your matcher makes use of.
- **It must be realistically messy.** A clean catalog makes matching trivial and tells us nothing. Yours should contain, at a minimum:

- two editions of the same book as separate entries
- the same title published under two different titles (for example a US and a UK edition)
- two genuinely different books that share a title
- an omnibus or collected edition alongside the individual volumes it contains
- titles that are substrings of other titles
- author names that appear in more than one form — initials, accents, transliterations, or `Lastname, Firstname` order
- **Weight it towards books people actually own.** At the presentation we will hand you photos of **our** shelves. If your catalog is 100 obscure titles, nothing will match and neither of us learns anything.

Don't spend long on this. Half an hour is plenty. Generate it with an LLM if you want to — that's what we'd do. But you own its quality, and we will ask you why you included what you included.

Four things we will specifically check

- 1. Matching against a messy catalog.** Exact string matching will fail on a catalog built to the spec above. Show us how you handle that, and how you arrive at a confidence score.
- 2. Local vs. hosted routing.** Be explicit about which work happens on the local model and which hits the vision-language model, and why. **Report measured per-image latency and estimated per-image API cost in your README.** Numbers, not adjectives.
- 3. Human in the loop.** Low-confidence results must reach the user for confirmation. They must not be silently accepted, and they must not be silently dropped. Treat the review step as part of the product, not as a debug screen.
- 4. Graceful failure.** A model timeout, malformed JSON back from the model, zero books detected, an unreadable spine — none of these should crash the app or leave the user staring at a blank screen.

Deliverables

- **A GitHub repository.** Public, or private with an invite to the handle we gave you.
- **Real commit history.** Incremental, meaningful commits. A single `initial commit` containing the whole project will count against you. No commits after you submit.
- `catalog.csv` — your catalog, built to the spec above.
- **The photos you tested with, committed to the repository.**
- **A README** containing:
 - setup and run steps that work from a clean clone

- a short architecture description
- your measured latency and cost numbers
- how you built your catalog and what ambiguity you deliberately put in it
- key decisions and the tradeoffs behind them
- what is unfinished, and what you would do with another day
- **A few real tests.** We want tests on the matching logic. We are not looking at coverage percentage.
- **AI_USAGE.md** — a short, honest note on which AI tools you used and where. Using AI heavily is expected and completely fine. Pretending you didn't is not.

The presentation — 30 minutes

- **10 minutes** — you demo it. We will hand you two or three of **our** bookshelf photos to run live.
- **10 minutes** — questions about your architecture and your AI decisions.
- **10 minutes** — we ask you to make one small change to the code, live.

What we're grading

- Judgment under a deadline — what you chose to build, and what you chose to skip
- Whether the pipeline shows real thinking about cost, latency, and failure
- Whether the matching logic is more than a string comparison
- Whether you understood what makes matching hard, and built a catalog that proves it
- Whether the interface respects the fact that the model is sometimes wrong
- Whether you can explain and modify every line in the repository

What we're not grading

Visual polish beyond "clean and usable." Authentication. Deployment. Test coverage numbers. Raw accuracy on difficult photos — we care that you **measured** it and **handled** it.

Notes

- **Ask questions.** Sending us a good clarifying question is a positive signal, not a negative one.
- If you run out of time, ship what works and say so. An honest README scores better than a broken feature presented as finished.